

SISTEMA DE GESTIÓN DE CONJUNTOS RESIDENCIALES

Documentación Técnica del Proyecto

Curso:

Redes e Infraestructura 2026-01

Autores:

Emmanuel Escobar Santos

Código: 2245212

emmanuel.escobar@uao.edu.co

Johann Eduardo Gonzalez Sandoval

Código: 2245329

johann_edu.gonzalez@uao.edu.co

Juan David Lasso Chaparro

Código: 2248450

juan_dav.lasso@uao.edu.co

Docente:

Oscar Hernan Mondragon Martinez

ohmondragon@uao.edu.co

Fecha:

22 de Mayo del 2026

Sistema de gestión de conjuntos residenciales

I. RESUMEN

Este documento presenta la arquitectura, diseño e implementación de **Redechitas**, una plataforma web para la gestión integral de conjuntos residenciales basada en microservicios. El sistema emplea cuatro microservicios independientes orquestados con **Docker Swarm**, balanceados con **HAProxy** y potenciados por un clúster distribuido de **Apache Spark** para analítica de datos. Se describen la problemática abordada, las decisiones tecnológicas, la arquitectura, los componentes, los pipelines de procesamiento y los procedimientos de prueba de rendimiento.

II. CONTEXTO Y PROBLEMÁTICA

La administración de conjuntos residenciales en Colombia enfrenta retos operativos críticos: gestión descentralizada de residentes y propiedades, registro manual de pagos de administración, control ineficiente de parqueaderos y torres, y ausencia de herramientas analíticas para la toma de decisiones. Los sistemas disponibles son en su mayoría monolíticos, costosos y difíciles de escalar ante demandas variables.

En el contexto del curso se identificó la necesidad de construir una plataforma que demostrara principios modernos de ingeniería: independencia de despliegue, escalabilidad horizontal, tolerancia a fallos y analítica de datos a gran escala.

III. SOLUCIÓN PROPUESTA

Redechitas es una plataforma web de microservicios donde cada servicio es autónomo en su ciclo de vida, base de datos y despliegue. Cuatro microservicios cubren los dominios del negocio residencial, expuestos a través de un proxy inverso HAProxy y consumidos por un frontend estático servido por Nginx, todo orquestado con Docker Swarm.

Servicio	Función	Tecnología	Puerto
MS1	Gestión de Usuarios	Node.js + Express	3000
MS2	Gestión de Propiedades	Node.js + Express	3001
MS3	Gestión y Pagos	Node.js + Express	3002
MS4	Análisis	FastAPI + PySpark	3003

Tabla I. Microservicios del sistema.

IV. DATASET SINTÉTICO

Para alimentar el microservicio de análisis, se generó un dataset sintético con registros ficticios que simulan la operación real de conjuntos residenciales.

Entidad	Registros
Usuarios	276
Torres	317
Parqueaderos	18
Pagos	6.466
Gestiones	461
Conjuntos	5
Apartamentos	677

Tabla II. Composición del dataset publicado en Kaggle.

V. COMPARATIVA: DESPLIEGUE Y ORQUESTACIÓN

Se evaluaron tres alternativas de orquestación de contenedores. Docker Swarm fue seleccionado por su integración nativa con Docker CLI, baja curva de aprendizaje, se alcanzó cierto dominio por lo aprendido en clase, y suficiente capacidad para el alcance del proyecto.

Criterio	Docker Swarm	Kubernetes	Nomad
Complejidad config.	Baja (CLI Docker)	Alta (YAML extenso)	Media (HCL)
Curva aprendizaje	Baja	Muy alta	Media
Escalabilidad	Buena (manual/auto)	Excelente (HPA)	Buena
Alta disponibilidad	Sí (restart_policy)	Sí (ReplicaSets)	Sí (restarts)
Balanceo nativo	DNS Round-Robin	kube-proxy	Consul+Fabio
Monitoreo integrado	Básico	Prometheus+Grafana	Nomad UI
Soporte ecosistema	Medio	Muy amplio	Limitado

Tabla III. Comparativa de orquestación de contenedores.

VI. COMPARATIVA: PROCESAMIENTO DE DATOS

Se evaluaron tres motores de procesamiento distribuido. Apache Spark fue seleccionado por su madurez, amplia API PySpark, soporte SQL/ML nativo y compatibilidad con el clúster standalone desplegado en Swarm.

Criterio	Apache Spark	Apache Flink	Dask
Paradigma	Batch + micro-batch	Streaming + Batch	Paralelo Python
API principal	Python/Scala/Java/R	Python/Java/Scala	Solo Python
Motor SQL	Spark SQL (maduro)	Flink SQL	Limitado
ML integrado	MLlib (completa)	FlinkML (básico)	Scikit-learn
Gestión clúster	Standalone/YARN/K8s	Standalone/K8s	No nativo
Overhead	Medio (master+workers)	Alto (JM+TM)	Bajo
Volúmenes grandes	Excelente	Excelente	Medio

Tabla IV. Comparativa de motores de procesamiento de datos.

VII. ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura de microservicios, donde cada servicio gestiona su propio ciclo de vida y base de datos MySQL. La comunicación entre servicios es REST/HTTP sobre una red overlay de Docker Swarm. El único punto de entrada externo es HAProxy en el puerto 8080.

El clúster Docker Swarm consta de un nodo manager y dos nodos worker. Bases de datos y Spark Master residen en el manager; los Spark Workers y el Frontend se distribuyen en workers.

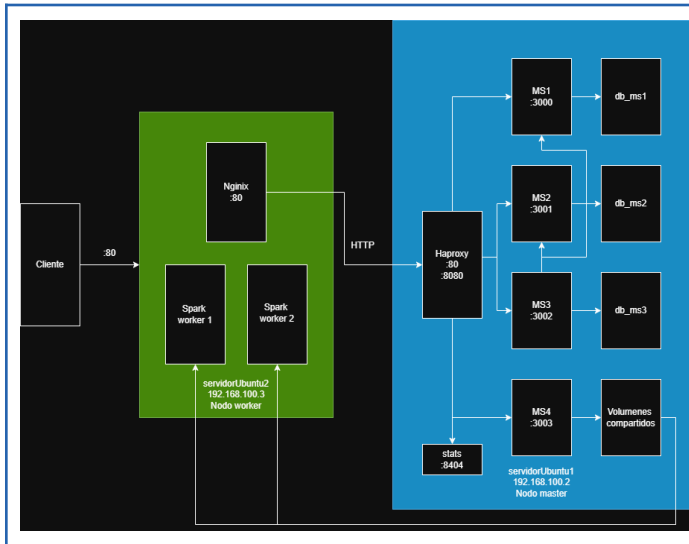


Figura 1. Diagrama de arquitectura.

A continuación se adjunta el diagrama de despliegue para una representación más técnica

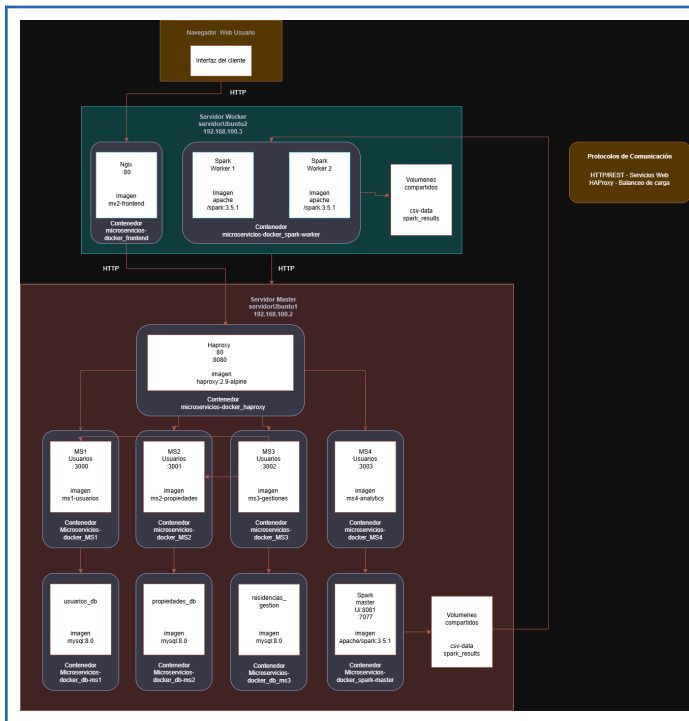


Figura 2. Diagrama de despliegue.

VIII. COMPONENTES, ALGORITMOS Y PIPELINE

A. MS1 – Gestión de Usuarios (puerto 3000)

Implementado en Node.js + Express. Expone CRUD sobre la entidad *usuario* con campos: nombre, teléfono, email, rol (*administrador*, *arrendatario*, *arrendador*), conjunto_id y credenciales bcrypt. Triggers MySQL garantizan que solo administradores porten username y password. Gestiona `/api/login` y el flujo de cambio de contraseña en primer ingreso.

B. MS2 – Gestión de Propiedades (puerto 3001)

Administra la jerarquía física: Conjuntos → Torres → Apartamentos → Parqueaderos. Controladores independientes por entidad. Valida integridad referencial mediante consultas SQL con JOINS antes de insertar registros dependientes. Base de datos: `propiedades_db`.

C. MS3 – Gestión y Pagos (puerto 3002)

Orquesta las gestiones y pagos de administración. Depende de MS1 y MS2 para validación cruzada vía *HTTP interno* (`MS1_URL`, `MS2_URL`) antes de persistir. Base de datos: `residencias_gestion`.

D. MS4 – Analytics con Apache Spark (puerto 3003)

Construido con FastAPI + PySpark 3.5.1. Opera en dos modos: (1) pre-computado: *spark_processor.py* lee CSVs, calcula estadísticas, frecuencias y análisis de nulos, y persiste JSON en *spark_results*; (2) on-demand: SparkSession singleton para tendencia temporal, correlación de Pearson (*pyspark.ml.stat.Correlation*) y cruce de datasets vía JOIN distribuido.

E. HAProxy – Proxy Inverso y Balanceador

Reglas ACL por prefijo de URL distribuyen tráfico en modo round-robin entre réplicas mediante *server-template* con resolución DNS dinámica del Swarm (127.0.0.11). Health-checks TCP cada 10s con umbrales 2 éxitos / 3 fallos. Dashboard de estadísticas disponible en `:8404/stats`.

F. Frontend Nginx y Bases de Datos MySQL

Nginx sirve páginas estáticas organizadas por módulo (*capasFront-ms1...ms4*). Tres instancias MySQL 8.0 independientes inicializadas con scripts SQL montados como Docker Configs. Healthcheck con *mysqladmin ping* y periodo de inicio de 150s.

G. Pipeline de Procesamiento Spark

Etapas	Descripción
1. Ingesta	Lectura de CSVs desde volumen.
2. Transformación	Cast de tipos, manejo de nulos, formateo de fechas.
3. Agregación	groupBy + agg (sum, mean, count) para estadísticas y frecuencias.
4. Correlación	VectorAssembler + Correlation.corr (Pearson) sobre columnas numéricas.
5. Persistencia	Resultados JSON escritos en spark_results.
6. Exposición	FastAPI sirve los JSON pre-computados

Tabla V. Etapas del pipeline de procesamiento de datos.

IX. INTERACCIÓN ENTRE COMPONENTES

El flujo de trabajo completo a través del sistema sigue el siguiente recorrido:

Interacción	Flujo de trabajo
Cliente → Frontend	El navegador solicita páginas HTML estáticas a Nginx (puerto 80).
Frontend → HAProxy	Las llamadas REST del JS van a HAProxy (:8080), que evalúa el prefijo de la URL y enruta al backend correspondiente.
HAProxy → MS1/2/3/4	Round-robin entre réplicas activas. Las réplicas fallidas son excluidas automáticamente via health-check.
MS3 → MS1 y MS2	Antes de registrar gestión o pago, MS3 valida usuario (MS1) y propiedad (MS2) mediante HTTP interno en la red overlay.
MS4 → Spark Cluster	MS4 lanza jobs via spark-submit al master y los workers distribuyen el cómputo.
Spark ↔ Volúmenes	Clúster Spark y MS4 comparten csv_data (entrada) y spark_results (salida JSON) via volúmenes Docker montados.
Swarm → Contenedores	El manager mantiene el estado de réplicas, reinicia fallos y gestiona placement entre nodos.

Tabla VI. Flujo de interacción entre los componentes del sistema.

X. DESCRIPCIÓN DETALLADA DE COMPONENTES

Componente	Función principal	Datos que maneja
MS1 – Usuarios	CRUD de usuarios; autenticación JWT + bcrypt; control de roles con triggers MySQL.	Tabla usuario: id, nombre, teléfono, email, rol, conjunto_id, username, password_hash, password_changed.
MS2 – Propiedades	Gestión jerárquica Conjunto→Torre→Apartamento→Parqueadero; validación FK.	Tablas conjunto, torre, apartamento, parqueadero con relaciones de llave foránea.
MS3 – Gestión/Pagos	Registro de solicitudes residenciales y cobros; validación cruzada con MS1 y MS2.	Tablas gestion (tipo, descripción, estado, usuario_id, apt_id) y pago (monto, fecha, estado).
MS4 – Analytics	Análítica batch pre-computada y on-demand con PySpark; exposición vía REST.	CSVs en csv_data (entrada); JSON pre-computados en spark_results (salida).
HAProxy 2.9	Punto de entrada único; enrutamiento ACL; balanceo round-robin; stats en :8404.	Health-checks TCP en memoria; configuración vía Docker Config.
Apache Spark 3.5.1	Motor distribuido Master + 2 workers.	Procesa CSVs vía DataFrame API. Escribe resultados JSON vía SparkSession.
Frontend – Nginx	Servidor de archivos estáticos; módulos por microservicio.	Token JWT manejado en localStorage via auth.js.
MySQL 8.0 (×3)	Persistencia independiente por servicio.	3 BDs aisladas: usuarios_db, propiedades_db, residencias_gestion. Init via Docker Configs.

Componente	Función principal	Datos que maneja
Spark Workers (×2)	Nodos de cómputo distribuido; escalables horizontalmente.	Procesan particiones de DataFrames; comparten volúmenes csv_data y spark_results.

Tabla VII. Descripción detallada de componentes, función y datos.

XI. PRUEBAS DE FUNCIONAMIENTO —

Se diseñaron 3 pruebas de cargas distintas con Apache JMeter para validar tres aspectos críticos: balanceo de carga de HAProxy, escalabilidad horizontal de los microservicios y resistencia de los contenedores bajo condiciones de estrés. Todos los escenarios se ejecutan contra el endpoint :8080 de HAProxy.

A. Escenarios de Prueba

Escenario	Descripción
Balanceo de carga	2 réplicas activas de MS1/MS2/MS3. Carga constante de usuarios concurrentes (1.000, 4.000, 10.000, 15.000, 20.000).
Escalabilidad horizontal	Se realizó la misma prueba anterior pero esta vez escalado con 4 réplicas.
Estrés de contenedores	Se realizaron 3 pruebas (500, 1.000, 2.000) cada una durante 3 minutos, mientras los usuarios entran gradualmente en los primeros 60 segundos para luego enviar peticiones infinitamente hasta terminar los 3 minutos.

Tabla VIII. Escenarios de prueba de carga.

B. Resultados – Balanceo de Carga

Los resultados obtenidos con la configuración base de dos réplicas por microservicio permitieron identificar con claridad el comportamiento del sistema ante distintos niveles de carga. Con 100 usuarios concurrentes el sistema respondió de manera fluida, manteniendo un tiempo de respuesta promedio de 1,131 ms y sin registrar ningún error, lo que nos confirmó que la arquitectura base es sólida para escenarios de uso normal según nuestro contexto. Sin embargo, al escalar a 400 usuarios el tiempo de respuesta se triplicó hasta los 3,794 ms, lo cual mostraba que la aplicación comenzaba a saturarse, aunque todavía no presentaba errores.

Al llegar a los 1,000 usuarios, empezaron a surgir los primeros errores (0.066%) y el tiempo máximo de respuesta alcanzó los 19,208 ms. A partir de ahí se comenzaron a hacer pruebas más exigentes a la aplicación web. Lo que más llamó la atención ocurrió entre 1,500 y 2,000 usuarios, donde el promedio de respuesta bajó de 3,777 ms a 2,380 ms, lo cual podría interpretarse como una mejora, pero en realidad es todo lo contrario. Esa caída en el promedio está acompañada de un tiempo máximo que escala hasta los 31,052 ms y de errores sostenidos cercanos al 0.61%, lo que indica que el sistema empezó a rechazar y descartar peticiones de forma rápida en lugar de procesarlas. Las respuestas instantáneas de error redujeron el promedio estadísticamente, pero esto ocultaba el hecho de que el sistema ya no estaba atendiendo las solicitudes de manera real.

Table 1: Resultados de Pruebas de Carga - JMeter (HTTP Request)												
Escenario	# Muestras	Promedio (ms)	Mín. (ms)	Máx. (ms)	Desv. Est.	Error (%)	Rendimiento (r/s)	Rec. KB/s	Err. KB/s	Bytes Prom.	Bytes Err.	Bytes Prom.
100 u., 1 s., >10	1000	1.131	21	2405	764.82	0.0000	73.98	41.98	8.96	581.0		581.0
400 u., 1 s., >10	4000	3.794	25	5.978	1.458.76	0.0000	96.22	54.59	11.65	581.0		581.0
1000 u., 1 s., >10	10000	4.785	0	19.208	2.458.27	0.0659	183.55	97.72	20.95	545.17		545.17
1500 u., 1 s., >10	15000	3.777	0	16.488	4.337.06	0.5894	235.31	140.79	25.22	601.84		601.84
2000 u., 1 s., >10	20000	2.380	0	31.052	7.787.11	0.6129	588.88	276.54	55.19	560.37		560.37

n = muestras; s = segundos; r = segundos de ramp-up; >10 = 10 iteraciones. Tiempo de respuesta en milisegundos (ms). Rendimiento expresado en solicitudes/segundo.

C. Resultados – Escalabilidad Horizontal

Al aumentar el número de réplicas a cuatro por microservicio, los resultados nos mostraron algo que inicialmente puede parecer intuitivo: el escalado no siempre mejora el rendimiento de forma proporcional ni inmediata. Con 100 y 400 usuarios los resultados fueron prácticamente iguales a los obtenidos con dos réplicas, e incluso con 400 usuarios el promedio fue ligeramente peor (4,013 ms frente a 3,794 ms). Esto tiene una explicación la cual es que: con una carga baja, tener más réplicas activas introduce un overhead adicional en HAProxy, que debe hacer health checks, distribuir conexiones y resolver el DNS interno para cuatro instancias en lugar de dos. Haciendo que el proceso sea más pesado para HAProxy, mostrándonos que no siempre es mejor tener más réplicas.

Al realizar la prueba con 1,500 usuarios, el tiempo de respuesta promedio cayó de 3,777 ms a 1,788 ms, una mejora del 53%, y el throughput casi se duplicó pasando de 235 a 469 solicitudes por segundo. Esto ocurrió porque en ese rango de carga el cuello de botella estaba en el microservicio mismo, específicamente en el event loop de Node.js, que al ser single-threaded tiene un límite de peticiones que puede manejar simultáneamente. Con dos réplicas cada instancia tomaba aproximadamente 750 usuarios concurrentes saturando su event loop, mientras que con cuatro réplicas esa carga se distribuía en 375 usuarios por instancia, permitiéndole responder con mayor comodidad.

No obstante, con 2,000 usuarios los resultados con cuatro réplicas fueron casi idénticos a los de dos réplicas, con errores similares y promedios comparables. Aunque se presentó una mejoría leve en las estadísticas se llegó a la conclusión de que entre más réplicas el sistema no iba a tener una mejora significativa en sus tiempos de respuesta ni en los porcentajes de error si no que solamente generaría un gasto mayor de recursos que no sería compensada como debería.

Table 1: Resultados de Pruebas de Carga con Escalado Horizontal (2 → 4 réplicas) – JMeter (HTTP Request)

Escenario	# Muestras	Promedio (ms)	Mín. (ms)	Máx. (ms)	Desv. Est.	Error (%)	Rendimiento (r/s)	Res. KB/s	Err. KB/s	Bytes Prom.
100 u., 1 s., <10	1,000	1,135	24	3,799	341.85	0.0000	77.46	44.07	9.40	361.00
400 u., 1 s., <10	4,000	4,013	52	5,917	1,049.22	0.0000	93.48	53.04	11.32	561.00
1000 u., 1 s., <10	10,000	4,489	0	9,968	3,380.03	0.1021	188.15	116.42	20.51	633.50
1500 u., 1 s., <10	15,000	1,788	0	8,221	2,061.07	0.3314	469.84	248.95	32.18	542.58
2000 u., 1 s., <10	20,000	2,519	0	25,839	3,148.92	0.6095	484.73	292.33	51.36	617.56

u. = usuarios; s. = segundos; <10 = 10 iteraciones. Escalado horizontal aplicado de 2 réplicas a 4 réplicas durante las pruebas. Tiempo de respuesta en milisegundos (ms). Rendimiento expresado en solicitudes/segundo.

D. Resultados – Estrés de Contenedores

La prueba de estrés se realizó principalmente con 500 usuarios entrando a la aplicación de forma progresiva durante los primeros 60 segundos para luego realizar peticiones de forma infinita durante un total de 3 minutos (180 segundos) el sistema procesó 37,265 solicitudes con un promedio de 3,233 ms y apenas un 0.02% de errores, demostrando que tiene capacidad para sostener cargas pesadas de forma continua sin comprometer la estabilidad de la aplicación web.

Al aumentar la cantidad de usuarios a 1,000 usuarios la situación cambió drásticamente. Se procesaron únicamente 19,585 solicitudes, es decir, menos que en el escenario anterior a pesar de tener más usuarios y el mismo tiempo de prueba. El tiempo de respuesta promedio subió a 7,664 ms y el throughput cayó a la mitad, situándose en 108 solicitudes por segundo. Estos números indican que las peticiones estaban siendo puestas en cola dentro del sistema y muchas de ellas expiraban antes de recibir respuesta, lo que redujo el total de muestras procesadas y empeoró notoriamente los tiempos.

Finalmente se realizó una prueba casi llevaba al límite con 2,000 usuarios la cual produjo los datos tuvo 275,770 muestras procesadas, un tiempo de respuesta promedio de apenas 914 ms y un throughput de 1,524 solicitudes por segundo. Estos números a simple vista parecen indicar un rendimiento excepcional, pero en realidad ocurre lo contrario. Lo que ocurrió fue un colapso total del sistema donde

HAProxy, al no poder enrutar las peticiones a los microservicios saturados, comenzó a rechazarlas directamente devolviendo errores HTTP de forma casi instantánea, sin que la solicitud consiguiera llegar siquiera al microservicio. Obteniendo el error más alto hasta el momento siendo este del 0.93%.

Table 1: Resultados de Pruebas de Estrés – JMeter (HTTP Request)

Escenario	# Muestras	Promedio (ms)	Mín. (ms)	Máx. (ms)	Desv. Est.	Error (%)	Rendimiento (r/s)	Res. KB/s	Err. KB/s	Bytes Prom.
500 u., ramp 60 s., dur. 180 s	37,265	3,233	7	17,811	3,060.79	0.0215	206.18	72.09	24.43	358.03
1000 u., ramp 60 s., dur. 180 s	19,585	7,664	0	28,647	5,308.51	0.1375	108.57	66.02	11.32	622.67
2000 u., ramp 60 s., dur. 180 s	275,770	914	0	41,422	2,747.30	0.9301	1,524.55	2796.09	41.80	1,876.06

u. = usuarios; ramp = tiempo de arranque (ramp-up); dur. = duración total. Tiempo de respuesta en ms. Rendimiento en solicitudes/segundo.

XII. Conclusiones

El desarrollo de Redechitas permitió validar en la práctica los principios fundamentales de los sistemas distribuidos modernos. La adopción de una arquitectura de microservicios, orquestada mediante Docker Swarm y complementada con Apache Spark para el procesamiento analítico, demostró ser una solución técnicamente coherente y escalable para el dominio de la gestión residencial.

La decisión de mantener dos réplicas como configuración base en lugar de cuatro (cantidad con la cual se hicieron las pruebas de carga de escalado horizontal) no fue arbitraria, sino que quedó respaldada directamente por los datos obtenidos y por el contexto con el cual trabajamos nuestro proyecto. Primero que todo nuestro proyecto al ser una gestión de unidades residenciales se estableció que a este solamente podrían ingresar los administradores de los conjuntos y el superadministrador a pesar de poseer cuatro tipos de usuarios diferentes, lo cual nos indica desde un principio que la aplicación web en situaciones normales no tendrá una carga muy pesada de usuarios estando al mismo tiempo en la página y segundo como se quedó demostrado en las pruebas de carga con escalado horizontal, escalar a cuatro réplicas no mejora el rendimiento de la aplicación web de forma significativa teniendo en cuenta que está realizando un gasto mayor de recursos, así que se terminó tomando la decisión de escalar cada microservicio solamente con dos réplicas ya que en el contexto planteado muestra un buen rendimiento y un gasto bajo de recursos.

La separación de responsabilidades en cuatro microservicios autónomos, cada uno con su propia base de datos, elimina los cuellos de botella y permite desplegar, escalar y actualizar cada componente de forma independiente. El balanceo de carga mediante HAProxy, con resolución DNS dinámica sobre la red overlay de Swarm, garantiza la distribución equitativa del tráfico entre réplicas y la recuperación automática ante fallos de contenedores.

La generación de un dataset sintético de más de 7.000 registros y su publicación en Kaggle evidenció la importancia de contar con datos representativos para validar pipelines de analítica distribuida. El microservicio MS4, construido sobre FastAPI y PySpark, demostró la viabilidad de integrar procesamiento batch y consultas dentro de la misma plataforma, ofreciendo estadísticas descriptivas, análisis de correlación y tendencias temporales con latencias mínimas gracias al uso de resultados pre-computados.

Finalmente, las pruebas de carga con Apache JMeter confirmaron la capacidad del sistema para sostener tráfico concurrente real, validar la escalabilidad horizontal y determinar los límites de estrés de los

contenedores, aportando métricas objetivas que respaldan las decisiones de diseño adoptadas a lo largo del proyecto.

El desarrollo de Redechitas representó un aporte significativo al crecimiento profesional y personal del equipo. Enfrentarse a decisiones reales de arquitectura como elegir entre tecnologías de orquestación, diseñar la comunicación entre servicios o construir un pipeline de datos desde cero fortaleció la capacidad de análisis crítico y la habilidad para resolver problemas complejos bajo restricciones concretas. La naturaleza distribuida del proyecto exigió además un alto nivel de colaboración, comunicación y organización entre los integrantes.

XIII. Funcionamiento

Video del funcionamiento del proyecto:

<https://youtu.be/dy3fNslWMnM>